MDPI

Article

Accelerating Metagenomic Identification of DNA Sequences Using Artificial Neural Networks

Patryk Gryz [1] and Robert Nowak [2*]

[1] Warsaw University of Technology, Institute of Computer System, Nowowiejska 15/19, 00-665 Warsaw, Poland; patryk.gryz.stud@pw.edu.pl
[2] Warsaw University of Technology, Institute of Computer System, Nowowiejska 15/19, 00-665 Warsaw, Poland; robert.nowak@pw.edu.pl
* Correspondence: robert.nowak@pw.edu.pl; Tel.: +48 502 659 456

Abstract

Background: The growing volume of DNA sequence data demands efficient metagenomic idetification. It provides the possibility to construct the tools with sustainability performance to monitor environmental conditions, risks related to organisms and pathogens. **Methods:** A Convolutional Neural Network (CNN) leveraging contrastive learning is used to select representative sequences, which improve computational efficiency. **Results:** We present Exquisitor, a CNN-based tool. Benchmarking against classical methods shows superior classification accuracy and competitive execution time, highlighting CNNs' scalability and precision for genetic taxonomy. **Conclusion:** The presented research highlights the potential of CNNs for improving performance of metagenomic identification including taxonomic classification.

Keywords: artificial neural networks; contrastive learning; DNA sequencing; metagenomic identification; sequence clustering, monitor enviromental conditions, sustainability

1. Introduction

The use of information about genetic material found in the environment, combined with sequence databases, enables metagenomic identification – the identification of organisms present in a given sample. With the development of next-generation sequencing methods [1], sequencing costs have decreased significantly, and the number of DNA sequences being analysed has increased [2]. Traditional tools like BLAST often require unacceptable time and computational resources due to explosive database growth [3]. Consequently, new methods are needed to enable efficient analysis using available resources.

This study aims to propose a fast, high-quality metagenomic identification method using an artificial neural network (ANN) to select representative genetic sequences. Instead of classifying every environmental sequence, our approach focuses on identifying a representative subset for identification, thereby reducing computational complexity while maintaining accuracy.

Historically, the Needleman-Wunsch algorithm [4] first enabled genetic sequence comparison. Later, Lipman and Wilbur [5] introduced a more efficient approach based on *k*-tuples (a *k*-mer generalisation). This was further advanced by the BLAST algorithm [6], which leveraged *k*-mers for efficient similarity searches in large databases.

Over the past 15 years, metagenomic identification has gained popularity alongside sequence database growth. Tools like MetaPhiAn [7] and mOTUs2 [8] leverage marker

Received:
Revised:
Accepted:
Published:

genes to compare species composition or identify markers unique to specific microbial strains.

Other techniques include Centrifuge [9], which uses the Burrows-Wheeler transform [10] and Ferragina-Manzini Index [11] for efficient sequence retrieval, and Kraken [12], which combines *k*-mers with indexed databases. Modern machine learning approaches include BERTax [13], treating DNA as a language via transformer models [14] for database-free classification, and CGRclust [15], which utilizes chaos game representation with convolutional neural networks.

Metagenomic identification currently relies on three main approaches: alignment, *k*-mers and marker genes. Alignment-based tools offer the highest accuracy but are computationally expensive. Algorithms based on *k*-mers provide high speed but lower quality by ignorning structural information. Marker gene solutions deliver good results but are limited to organisms with known reference markers.

Although recent studies use machine learning for metagenomic identification, research on artificial neural networks (ANN) with contrastive learning for sequence clustering remains limited. ANNs often outperform classical heuristics, offering faster and more accurate results. This works leverages ANN capabilities to accelerate metagenomic identification while maintaining traditional quality levels, significantly streamlining the classification process.

The rest of the paper is organized as follows: Section 2 defines the problem, the proposed approach, and the Exquisitor tool. Section 3 describes the experimental setup, datasets, performance metrics, and results. Finally, Section 4 summarises the findings, discusses limitations, and suggests future research directions.

2. Methods

The core component of the proposed tool, Exquisitor, is a processing pipeline designed with flexibility in mind, allowing the integration of various DNA sequence clustering methods. The pipeline consists of a series of steps executed sequentially to perform metagenomic identification. A schematic representation of the pipeline is show in Fig. 1, where input and output data are represented by circles, and a rectangle represents processing step. The pipeline is composed of the following five steps:

1. **Preparation of DNA Sequences**
   This step verifies DNA sequence correctness and aligns them to a required length for subsequent processing.
2. **Dissimilarity Calculation**
   Dissimilarity between sequences is calculated using two classical methods or one custom approach, producing a dissimilarity matrix for clustering.
3. **Sequence Clustering**
   Sequences are grouped into clusters, each with one representative and several members. This reduces the data volume for later stages by focusing on the most informative representatives.
4. **Database Search**
   Representative sequences are queried against a reference database to identify similar sequences and infer their taxonomic origin.
5. **Postprocessing**
   This final step integrates cluster information with the database search results to produce the final identification.

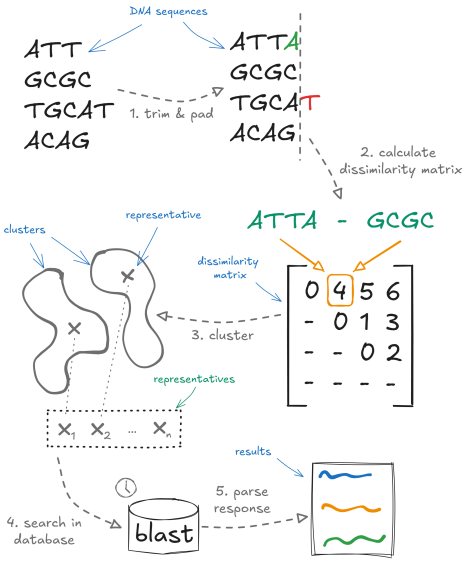

Figure 1. Overview of the pipeline: preparation of DNA sequences, dissimilarity calculation, sequence clustering, database search and postprocessing.

2.1. Exquisitor: Method Description

The proposed method uses an artificial neural network to reduce the dimensionality of input DNA sequences to a feature vector in the form of $\mathbb{R}^{64}$. The neural network employs contrastive learning, enabling representations to be learned while preserving dissimilarity between sequences. The dissimilarity between sequence representations will be calculated using cosine dissimilarity, expressed by the formula in equation (1).

$$dsim(A, B) = 1 - sim(A, B) \tag{1}$$

where:

$$sim(A, B) = \cos\theta = \frac{A \cdot B}{\|A\|\|B\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \cdot \sqrt{\sum_{i=1}^{n} B_i^2}}$$

A, B – compared vectors,

θ – angle between A and B,

A_j, B_j – j-th element of the vector A and B, respectively.

2.1.1. Artificial Neural Network Architecture

The ANN model consists of two parts. The first part comprises two convolutional blocks with batch normalisation and is responsible for feature extraction from the sequence. The second part utilises fully connected layers with dropout and the GELU activation function and is connected to the first part of the model via a flattening layer. The model output is the output of the final linear layer, which is a feature vector of dimension $\mathbb{R}^{64}$.

Schematically, the architecture is presented in Fig. 2.

Design Rationale

Convolutional Neural Networks (CNNs) were chosen over Recurrent Neural Networks (RNNs) for faster training and inference. Contrastive learning allows the model to measure sequence similarity, which supports clustering.

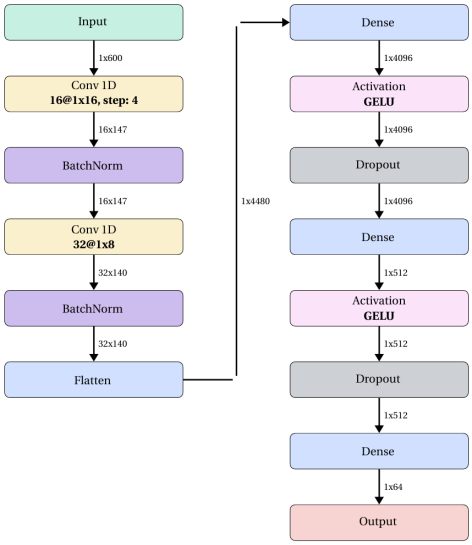

Figure 2. Diagram of ANN architecture. Two parts were depicted: the convolutional blocks (on the left) are responsible for feature extraction, and fully connected layers with dropout to generate the results.

Input

The input to the model consists of DNA sequences of length 150[1], which are encoded into a vector of dimensions 1×600 using one-hot encoding [16].

Training Examples

The training and validation examples consist of an anchor, a positive sequence similar to the anchor, and a negative sequence dissimilar to the anchor.

Dataset

The training and validation datasets were created based on the first sample of genetic sequences from the CAMI II Toy Human Microbiome Project dataset [17]. The sample contains simulated metagenomic data from the human skin microbiome. Examples were obtained by randomly selecting anchors from the dataset and modifying these anchors to create positive and negative sequences. The modification involved a point mutation of a given nucleotide to another one. In the case of positive sequences, the change affected between 0% and 20% of the anchor length, whereas for negative sequences, the change ranged from 20% to 80% of the anchor length.

Loss function

The loss function used is defined as:

$$\text{Contrastive loss} = [m_{pos} - s_{pos}]_+ + [s_{neg} - m_{neg}]_+ \quad (2)$$

[1] The sequence length of 150 is a method parameter and can be adjusted depending on the application.

where:

m_{pos} – similarity margin between the positive sequence and the anchor,

m_{neg} – similarity margin between the negative sequence and the anchor,[2]

s_{pos} – cosine similarity of the positive sequence to the anchor,

s_{pos} – cosine similarity of the negative sequence to the anchor.

The proposed loss function is similar to the triplet neural network framework [18], but it is adapted to use cosine dissimilarity.

Learning

The model was trained on a dataset of 10^6 training examples and 10^4 validation examples. The optimiser used was *AdamW*, and the training was performed for 100 epochs. Both weight decay and the exponential learning rate schedule were employed to mitigate the risk of overfitting. The convergence criterion was defined as the absence of further improvement in the validation loss.

Quality

As a measure of model quality, the contrastive loss of the model was computed on the validation set.

Parameters

Experiments were conducted to determine the optimal training parameters for the artificial neural network model. The tested parameters included the learning rate coefficient λ, weight decay w, the coefficient γ used in the exponential learning rate decay, the dropout rate, and the effectiveness of using three perceptron layers. The parameter search was performed using grid search.

As a result of these experiments, the best parameters were selected:

$$\lambda = 10^{-6}, \quad w = 10^{-4}, \quad \gamma = 0.99999$$

a dropout rate of 0.5, a batch size of 256, and a confirmed positive impact of using three perceptron layers.

Additionally, the loss function was used with the following parameters:

$$m_{pos} = 1.0, \quad m_{neg} = 0.25.$$

Results of Learning

As a result of training the neural network, the model achieved a validation set loss of 0.17.

2.2. Exquisitor: Implementation Details

The solution consists of two components. The first is a library that contains components enabling the creation of pipelines for genetic material analysis in the form of metagenomic identification. The second component is a console application, which uses

the library's components to create and configure processing pipelines. A web application was also created, allowing users to submit analysis tasks through a web browser.

The console application was implemented using the *clap* library. In the library the *k*-medoid algorithm was provided by the *kmedoids* library [19], and the neural network was built using the *burn* library with the *wgpu* execution environment. Web application was implemented using the *axum* library with *tokio* async environment.

Additionally, the following tools were used in the work:

- *cargo* as the package manager and build system in Rust;
- *rustup* for automatic management of Rust versions;
- *clippy* for static code analysis in Rust;
- *rustfmt* for automatic formatting of Rust source code;
- *cargo test* for conducting unit tests;
- *git* as the version control system, enabling change tracking and code history management.

3. Results

This section presents the results of the experiments conducted to evaluate the proposed solution (Exquisitor), which aims to improve performance while maintaining the quality of metagenomic identification. The study compared the new approach with two classical methods using the same dataset. The evaluation focused on execution time and identification quality to assess the effectiveness of the proposed approach.

3.1. Datasets

3.1.1. Dataset Description

Experiments utilized the textitCAMI II Toy Human Microbiome Project dataset [17], a dataset created for bioinformatics benchmark. Dataset was used also in ANN training. For testing, disjoint subsets were created with sizes 2^k for $k \in [0, 12]$, exclusively using sequences not seen during the training phase to ensure unbiased evaluation.

3.1.2. Dataset Preparation

The subsets were created by randomly sampling, without replacement, the indices of genetic sequences from the reference dataset that were to be included in each subset. Indices of sequences used in the ANN training and validation sets were excluded from the sampling. The sizes of the subsets were as follows: training set - 1,000,000 sequences, validation set - 10,000 sequences, and test set - 8,192 sequences.

3.2. Clustering Algorithm

In the experiments, we used an external k-medoids clustering algorithm [19]. The medoids identified by this algorithm were used as cluster representatives and used in subsequent steps.

3.3. Base-line Algorithms

Two classical approaches were implemented and used to compare their results with the results of the proposed neural network.

3.3.1. Modified Needleman-Wunsch Algorithm

The first classical approach is the Needleman-Wunsch algorithm. This algorithm allows for global alignment of genetic sequences. It uses a similarity matrix between sequences, which is constructed according to the rules given in equation (3). A modification of the approach presented in equation (4) is applied in the work.

$$
\begin{aligned}
D_{i,0} &= i \cdot g, && \text{for } i \in [1, n+1] \\
D_{0,j} &= j \cdot g, && \text{for } j \in [2, m+1] \\
D_{i,j} &= \max \begin{cases} D_{i-1,j} + g \\ D_{i,j-1} + g \\ D_{i-1,j-1} + s(A_i, B_j) \end{cases}, && \text{for } i \in (1, n+1] \text{ and } j \in (1, m+1]
\end{aligned} \tag{3}
$$

where:

A, B – compared sequences,
n, m – lengths of sequences A and B,
D – similarity matrix of size $n+1$ x $m+1$,
$g \in \mathbb{R}$ – penalty for a gap,
$s(A_i, B_j) \in \mathbb{R}$ – similarity between the i-th element in sequence A and the j-th element in sequence B.

$$
\begin{aligned}
D_{i,j} &= \min \begin{cases} D_{i-1,j} + g \\ D_{i,j-1} + g \\ D_{i-1,j-1} + s(A_{i-1}, B_{j-1}) \end{cases}, \\
&\text{for } i \in (1, n+1] \text{ and } j \in (1, m+1]
\end{aligned} \tag{4}
$$

In the solution, the following parameter values were adopted:

$$g = 2,$$

$$s(a, b) = \begin{cases} 0, & \text{for } a = b, \\ 1, & \text{for } a \neq b. \end{cases}$$

3.3.2. *k*-mer Embeddings

The second approach use *k*-mers as embeddings, which allows for the representation of DNA sequences as numerical vectors. Embedding is created by constructing a vector of size 4^k, where each *k*-mer corresponds to a specific position in the vector. The number of occurrences of each *k*-mer in the sequence is then recorded at its corresponding position. These vectors will then be compared using Euclidean distance.

3.4. Experiments

3.4.1. Execution Time of Metagenomic Identification with Exquisitor

Objective

Measuring the execution time of metagenomic identification using different genetic sequence clustering methods.

Assumptions

1. Sequences clustering is deterministic with a fixed seed.
2. Metagenomic identification is the only task running on the machine.

Table 1. Metagenomic identification execution time comparison of presented solution (NP) and benchmark methods (NW, *k*-mer, ANN).

Number of sequences	Execution time [s]			
	NP	**NW**	***k*-mer**	**ANN**
1	7107	**6025**	6087	6077
2	6129	5994	**5988**	6035
4	6108	**5983**	6024	6019
8	6196	5988	**5925**	6014
16	6290	**5941**	5946	6118
32	6279	6035	**5962**	6092
64	6316	6113	**6107**	6139
128	6560	6238	**6206**	6233
256	6753	6446	**6295**	6363
512	7091	6972	**6326**	6347
1024	8059	8743	**6248**	6269
2048	8517	16461	6472	**6332**
4096	9433	**timeout**	7003	6550

Results

Experimental execution times were recorded for all methods, though the modified Needleman-Wunsch (NW) approach failed to complete for 4096 sequences due to time constraints. Fig. 3 illustrates execution time as a function of input sequence count for: the modified Needleman-Wunsch algorithm (NW), *k*-mer embeddings (*k*-mer), artificial neural networks (ANN), and a no-pipeline baseline (NP). Detailed timing data are presented in Table 1.

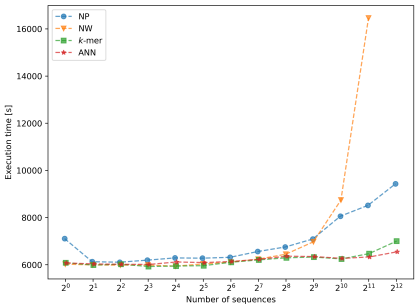

Figure 3. The comparison of the metagenomic identification execution time by the presented solution (NP) and benchmark methods: modified Needleman-Wunsch algorithm (NW), *k*-mer embeddings (*k*-mer), and a method using an artificial neural network (ANN).

Conclusions

The modified Needleman-Wunsch (NW) approach exhibited the fastest execution time increase due to its computationally intensive sequence comparisons. Despite reducing the number of classified sequences, its total runtime exceeded the no-pipeline baseline. In contrast, *k*-mer embeddings and ANN showed comparable, significantly faster execution times by using sequence representations for dissimilarity matrix construction. For 4096 sequences, the ANN benefited from GPU-accelerated embedding computations, further optimizing performance compared to traditional metagenomic identification.

3.4.2. Quality of Metagenomic Identification with Exquisitor

Objective

Examining the quality of metagenomic identification using the implemented methods in comparison to the metagenomic identification of all sequences.

Assumptions

1. Sequences clustering is deterministic with a fixed seed.
2. `BLASTn` is deterministic and always returns the same results for a given sequence and specified parameters.
3. In the case where it is not possible to compute the quality for a given metagenomic identification run, a quality value of 0 is assumed. This value is not considered when calculating the weighted average quality.

Exquisitor Performance Measures

Quality of metagenomic identification was measured using a modified Jaccard index, expressed by equation (5), as standard classification metrics relying on false positives/negatives are not well-defined when BLAST results serve as the reference. The measure was used to compare the quality of metagenomic identification performed using the implemented methods against metagenomic identification without using a processing pipeline, i.e., using raw BLAST results.

$$\mathrm{Q} = \frac{\sum_{r \in (O(R) \cap O(E))} R_r}{\sum_{r \in (O(R) \cup O(E))} R_r} \tag{5}$$

where:

$$\begin{aligned} R &- \text{set of reference results,} \\ E &- \text{set of obtained results,} \\ R_r &- \text{number of results in reference set} \\ &\quad \text{assigned to the organism } r \\ O(X) &- \text{set of unique organisms, for which results} \\ &\quad \text{were assigned in the set } X \end{aligned}$$

The weighted average quality, defined by equation (6), was used to compare the quality of multiple metagenomic identification runs.

$$Q_{\text{avg}} = \sum_{c \in C} \frac{n_c}{n} Q_c \tag{6}$$

where:

$$\begin{aligned} C &- \text{set of metagenomic identification runs,} \\ Q_c &- \text{quality of metagenomic identificationn } c, \\ n_c &- \text{number of input sequences for metagenomic} \\ &\quad \text{identification } c, \\ n &- \text{number of input sequences } n = \sum_{c \in C} n_c. \end{aligned}$$

Results

Classification quality was calculated using equation (5) relative to full metagenomic identification. Fig. 4 and Table 2 present these results as a function of the input sequence

Table 2. Comparison of the metagenomic identification quality returned by the presented method with the benchmark methods.

Number of sequences	Method		
	NW	***k*-mer**	**ANN**
1	**1.0**	**1.0**	**1.0**
2	0.27	**0.73**	**0.73**
4	**0.8**	0.2	0.2
8	**0.94**	0.88	0.12
16	0.82	0.82	**0.94**
32	0.88	**0.92**	0.87
64	0.93	**0.94**	0.93
128	0.89	0.87	**0.91**
256	0.89	0.89	**0.93**
512	0.73	0.76	**0.94**
1024	**0.95**	0.93	**0.95**
2048	0.92	**0.95**	0.92
4096	0.0	0.93	**0.96**

count. The weighted average quality (Eq. (6)) was 0.899 for the modified Needleman-Wunsch algorithm, 0.921 for the *k*-mer embedding method, and 0.944 for the artificial neural network approach.

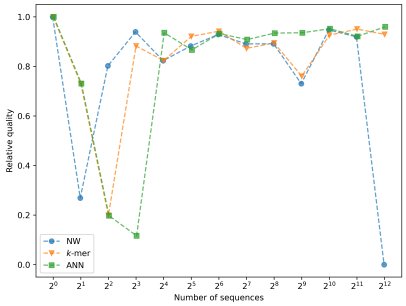

Figure 4. Comparison of the metagenomic identification quality returned by the presented method with the benchmark methods.

Conclusions

All implemented methods achieved high classification quality, exceeding 0.7 when the number of groups significantly exceeded input sequences. The artificial neural network (ANN) approach yielded the highest weighted average quality, slightly outperforming both *k*-mer embedding and the modified Needleman-Wunsch algorithm. This superior performance stems from the model's ability to consider the full sequence structure, enabling more precise dissimilarity determination.

3.4.3. Performance on an unseen dataset

Objective

Assessing the quality of clustering on unseen data from a different sample.

Table 3. Comparison of clustering quality on seen and unseen data.

Sample	AMI	NMI	NMI
Sample 1 (seen)	0.288	0.286	0.424
Sample 4 (unseen)	0.219	0.243	0.379

Preparation

We obtained an additional sample from a different body site (CAMI II Toy Human Microbiome Project[3]). Processing steps were applied both to the training samples and to this new sample:

1. Sequences in each sample were mapped to taxonomic identifiers (tax_id).
2. From each sample, a subset of 10,000 sequences was randomly selected, ensuring that the dominant taxon did not exceed 50% of the subset.
3. Sequence embeddings were generated using the ANN model.
4. For each sample, clustering was performed with the k-medoids algorithm, where k was chosen according to the number of true taxa in the subset.
5. Clusters corresponding to taxa were treated as the ground truth.

The clustering performance was evaluated by computing Adjusted Mutual Information (AMI), Normalised Mutual Information (NMI), and the Fowlkes–Mallows Index (FMI) between the obtained clusters and the ground truth.

Results

The evaluation metrics (AMI, NMI, FMI) for clustering performance are summarised in Table 3.

Conclusions

Although the clustering performance on the unseen data is not optimal, it remains comparable to the results obtained for the training data. It suggests that the model's generalisation capabilities are present but constrained, and that the outcomes are not merely random.

4. Discussion

4.1. Interpretation of Results

Experimental results partially confirmed initial expectations, as all methods achieved satisfactory identification quality. The ANN-based approach performed best, yielding the highgest weighted average classification quality with execution time comparable to k-mer embeddings. Homever, small input samples led to lower quality, likely due to poor representative selection; thus the method is recommended for larger datasets. For the testsed data, the efficiency threshold was 128 sequences, where quality remained high and reduced processing time by 5%. The ANN no only improved classification quality over the methods but also remained competitive with classifical approaches.

Our pipeline increases the speed of BLAST by using a single sequence that represents a cluster, and it uses BLAST internally. The presented method is not limited to metagenomic analysis; generally, it replaces BLAST in tasks when clusters of sequences are analysed. Therefore, genome assembly, identifying cell subtypes, and phylogenetic trees might benefit from the presented approach.

[3] Sample 4, airways

4.2. Limitations

4.2.1. Experiments Duration

The full metagenomic identification of DNA sequences for each method and each experimental subset took approximately 5 days, which led to the limitation of the number of experiments to a single run and subset size to 4096 sequences.

4.2.2. Single Dataset

Using a single human skin microbiome sample likely constrained the sequence space and influenced results, as both the ANN and experiments shared the same source. Furthermore, the lack of duplicate checking might have caused data leakage, potentially biasing the evaluation.

4.2.3. Single Dissimilarity Matrix

Using a single dissimilarity matrix for all sequences limits the number of input sequences, as the matrix build time scales quadratically with the number of sequences.

4.2.4. Baseline Methods Limitations

We used simplified baseline methods, which do not account for different gap penalties as implemented in algorithms such as Gotoh [20], nor do they distinguish between different nucleotide substitutions, such as transitions and transversions.

4.2.5. Model Limitations

The model's generalization is restricted to human skin microbiome data; other sample types require further study. While the ANN benefits from GPU acceleration, CPU-only execution significantly increases runtime. Additionally, the approach is optimized for sequences of similar length, showing limited adaptability to variation. Finally, using CNNs instead of RNNs or transformers may limit flexibility, though it ensures faster training and inference.

4.3. Possible Improvements

4.3.1. Architecture of Artificial Neural Network

The current ANN architecture lacks flexibility, requiring sequences of similar lengths. Replacing the initial convolutional layers with Recurrent Neural Networks (RNNs) or transformers would enable processing of varying sequence lengths. In such a modification, these architectures would generate initial embeddings, which linear layers would then process to preserve dissimilarity properties.

4.3.2. Batching Sequences

Creating a single dissimilarity matrix for large samples is suboptimal due to its performance impact. Pre-clustering sequences into batches addresses this, maintaining short execution times at the cost of a potential quality reduction.

5. Conclusions

5.1. Summary of Findings

The experiments demonstrated that the proposed method based on artificial neural networks outperforms traditional approaches in terms of quality, while maintaining execution times comparable to those of the classical *k*-mer method. It highlights the potential of neural networks for improving performance without significant computational overhead.

5.2. Future Research Directions

Future research could focus on utilising artificial neural networks as an independent tool for metagenomic identification. Another promising direction is the application of artificial neural networks for dimensionality reduction of genetic sequences, which may improve efficiency and performance in various analyses. Planned development efforts are focused on providing interfaces for the Exquisitor tool, enabling its integration with existing popular metagenomic pipelines.

Author Contributions: Conceptualization, P.G. and R.N.; methodology, P.G.; software, P.G.; validation, P.G.; resources, R.N.; data curation, P.G.; writing, P.G and R.N.; supervision, R.N.; funding acquisition, R.N. All authors have read and agreed to the published version of the manuscript.

Funding: This work is supported by Statutory Resarch Grant of Institute of Computer Science, Warsaw University of Technology

Data Availability Statement: Project webpage with data https://github.com/pfgryz/exquisitor.

Acknowledgments: During the preparation of this manuscript/study, the author(s) used Grammarly and Gemini for spelling error correction. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Reinartz, J.; Bruyns, E.; Lin, J.Z.; Burcham, T.; Brenner, S.; Bowen, B.; Kramer, M.; Woychik, R. Massively parallel signature sequencing (MPSS) as a tool for in-depth quantitative gene expression profiling in all organisms. *Briefings in Functional Genomics* **2002**, *1*, 95–104.
2. Muir, P.; Li, S.; Lou, S.; Wang, D.; Spakowicz, D.J.; Salichos, L.; Zhang, J.; Weinstock, G.M.; Isaacs, F.; Rozowsky, J.; et al. The real cost of sequencing: scaling computation to keep pace with data generation. *Genome Biology* **2016**, *17*, 53.
3. Kodama, Y.; Shumway, M.; Leinonen, Rasko, o.b.o.t.I.N.S.D.C. The sequence read archive: explosive growth of sequencing data. *Nucleic Acids Research* **2011**, *40*, D54–D56, [https://academic.oup.com/nar/article-pdf/40/D1/D54/9482007/gkr854.pdf]. https://doi.org/10.1093/nar/gkr854.
4. Needleman, S.B.; Wunsch, C.D. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology* **1970**, *48*, 443–453.
5. Wilbur, W.J.; Lipman, D.J. Rapid similarity searches of nucleic acid and protein data banks. *Proceedings of the National Academy of Sciences* **1983**, *80*, 726–730.
6. Altschul, S.; Gish, W.; Miller, W.; Myers, E.; Lipman, D. Basic local alignment search tool. *Molecular Biology* **1990**, *215*, 403–410.
7. Segata, N.; Waldron, L.; Ballarini, A.; in.. Metagenomic microbial community profiling using unique clade-specific marker genes. *Nature Methods* **2012**, *9*, 811–814.
8. Milanese, A.; Mende, D.; Paoli, L.; in.. Microbial abundance, activity and population genomic profiling with mOTUs2. *Nature Communications* **2019**, *10*, 1014.
9. Kim, D.; Song, L.; Breitwieser, F.P.; L., S.S. Centrifuge: rapid and sensitive classification of metagenomic sequences. *Genome Research* **2016**, *26*, 1721–1729.
10. Burrows, M.; Wheeler, D.J. A Block-sorting Lossless Data Compression Algorithm. In Proceedings of the DEC SRC Research Report 124, 1994.
11. Ferragina, P.; Manzini, G. Opportunistic data structures with applications. In Proceedings of the Proceedings 41st Annual Symposium on Foundations of Computer Science, 2000, pp. 390–398.
12. Wood, D.E.; Salzberg, S.L. Kraken: ultrafast metagenomic sequence classification using exact alignments. *Genome Biology* **2014**, *15*, R46.
13. Mock, Florian AMD Kretschmer, F.; Kriese, A.; Böcker, S.; Marz, M. Taxonomic classification of DNA sequences beyond sequence similarity using deep neural networks. *Proceedings of the National Academy of Sciences* **2022**, *119*, e2122636119.
14. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, L.; Polosukhin, I. Attention is All you Need. In Proceedings of the Advances in Neural Information Processing Systems; Guyon, I.; Luxburg, U.V.; Bengio, S.; Wallach, H.; Fergus, R.; Vishwanathan, S.; Garnett, R., Eds. Curran Associates, Inc., 2017, Vol. 30.
15. Alipour, F.; Hill, K.; Kari, L. CGRclust: Chaos Game Representation for twin contrastive clustering of unlabelled DNA sequences. *BMC Genomics* **2024**, *25*, 1214.

16. Harris, D.; Harris, S. *Digital Design and Computer Architecture: From Gates to Processors*, 1 ed.; Elsevier Science & Technology: Burlington, 2007; p. 123.
17. Fritz, A.; Hofmann, P.; Majda, S.; in.. CAMISIM: simulating metagenomes and microbial communities. *Microbiome* **2019**, *7*, 17.
18. Hoffer, E.; Ailon, N. Deep metric learning using Triplet network, 2018, [arXiv:cs.LG/1412.6622].
19. Schubert, E.; Lenssen, L. Fast k-medoids Clustering in Rust and Python. *Journal of Open Source Software* **2022**, *7*, 4183.
20. Gotoh, O. An improved algorithm for matching biological sequences. *Journal of Molecular Biology* **1982**, *162*, 705–708.